

人工智能基础与应用 · 大模型实训

课题全集-11 个课题

人工智能基础与应用 · 大模型实训

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

目录

目录	2
课题 01 开题简报 · BPE 分词器	5
一、研究背景（这个问题历史上卡过谁）	5
二、研究问题（一句话，开放问题）	5
三、攻关线索（路标，不是答案）	5
四、里程碑（可勾选，3-6 个）	5
五、交付物要求	6
五之二、官方判分标准（2026-09-17 实跑探明，先知道标准再动手）	6
六、开题批（开场直接发给用户，3-5 题）	6
七、讲课锚点（知行合一的“讲”这一腿）	7
八、参考实现（只准用于对照，不准抄）	7
判卷记录 · 课题 01 BPE 分词器	8
L1 · 基线（2026-09-17 立科时实跑）	8
L2 · 导师审稿意见	8
L3 · 费曼收口	8
本课题的判分环境坑（已解决，记录备查）	8
课题 02 开题简报 · Transformer 骨架与训练循环	10
一、研究背景	10
二、研究问题（一句话）	10
三、攻关线索	10
四、里程碑	10
五、交付物	10
六、讲课锚点（讲义 M3 + 对标）	11
七、代码级提问示例（讲课中出，先预测后验证）	11
八、风险与提醒	11
课题 03 开题简报 · Scaling Law 预演与打脸链路	12
一、研究背景	12
二、研究问题（一句话）	12
三、攻关线索	12
四、里程碑	12
五、交付物	12
六、讲课锚点	12
七、代码级提问示例	13
八、风险与提醒	13
课题 04 开题简报 · 0.1B 端到端预训练（首次上机）	14
一、研究背景	14
二、研究问题（一句话）	14
三、攻关线索	14

四、里程碑	14
五、交付物	14
六、讲课锚点	15
七、代码级提问示例	15
八、风险与提醒	15
课题 05 开题简报·手写 Triton attention kernel	16
一、研究背景	16
二、研究问题（一句话）	16
三、攻关线索	16
四、里程碑	16
五、交付物	16
六、讲课锚点	17
七、代码级提问示例	17
八、风险与提醒	17
课题 06 开题简报·多卡与分布式训练	18
一、研究背景	18
二、研究问题	18
三、攻关线索	18
四、里程碑	18
五、交付物	18
六、讲课锚点	18
七、代码级提问示例	19
八、风险与提醒	19
课题 07 开题简报·数据清洗与去重	20
一、研究背景	20
二、研究问题	20
三、攻关线索	20
四、里程碑	20
五、交付物	20
六、讲课锚点	20
七、代码级提问示例	21
八、风险与提醒	21
课题 08 开题简报·SFT / DPO / RLVR 三方对照	22
一、研究背景	22
二、研究问题	22
三、攻关线索	22
四、里程碑	22
五、交付物	23
六、讲课锚点	23
七、代码级提问示例	23
八、风险与提醒	23
课题 09 开题简报·可验证环境 + harness	24

一、研究背景	24
二、研究问题	24
三、攻关线索	24
四、里程碑	24
五、交付物	24
六、讲课锚点	25
七、代码级提问示例	25
八、风险与提醒	25
课题 10 开题简报·长程 Agent 与 self-judge	26
一、研究背景	26
二、研究问题	26
三、攻关线索	26
四、里程碑	26
五、交付物	26
六、讲课锚点	27
七、代码级提问示例	27
八、风险与提醒	27
课题 11 开题简报·结项（NeurIPS 报告 + 现场 demo）	28
一、研究背景	28
二、研究问题（结项版）	28
三、攻关线索	28
四、里程碑	28
五、交付物	28
六、讲课锚点	28
七、自检清单（结项前必过，抄自《判分与验收标准》§ 五）	29
八、风险与提醒	29

课题 01 开题简报 · BPE 分词器

模块：M2 分词与表示 | 周次：W1 (09-14→09-20) | 算力：T0 纯 CPU, 0 GPU·小时 唐杰作业对应：①
从零写 Tokenizer 的前半 | 判分来源：CS336 A1 tests/test_tokenizer.py + test_train_bpe.py 手写/
代写：本课题全部核心代码属手写区 R1 (见《造轮子边界.md》)

一、研究背景（这个问题历史上卡过谁）

- 2017 年前的 NLP 用整词表，未登录词 (OOV) 是死穴；字符级建模又让序列变得极长。
- BPE 从 2016 (Sennrich 等) 的机器翻译子词切分技术，被 GPT-2 (2019) 引入到字节级，从此成为大模型的事实标准 (GPT-2/3/4、LLaMA、Qwen、GLM 都是 BPE 系)。
- 关键洞察：词表不是“词汇表”，而是一个可学习的压缩算法——它同时决定了 ①模型的输入长度 ②嵌入层参数量 ③模型能见到的最小语义单元 ④多语言/代码/数学的公平性。

二、研究问题（一句话，开放问题）

在固定词表预算下，如何设计一个训练-编码两阶段都正确、且对中文/英文/代码都公平的子词分词器？——
并回答：为什么“字节起步”是零预算下唯一不会翻车的选择？

三、攻关线索（路标，不是答案）

1. 压缩视角：如果词表大小为 V ，BPE 训练在最大化什么？把它写成“每一步减少多少 token 数”。
2. 字节起步的理由：从 Unicode 字符起步会在哪些具体输入上崩？（想 3 个真实例子再往下走）
3. 训练算法：朴素实现为什么是 $O(n^2)$ ？哪些数据结构能把它压到近似线性？（提示方向：不要用字符串拼接）
4. 并列与终止：最高频并列时怎么办？什么时候停止合并？特殊 token (`<|endoftext|>`) 为什么不能参与合并？
5. 编码与解码：encode 的时候，遇到未在 merge 表里的相邻对怎么处理？为什么必须保证 `decode(encode(x)) == x` (往返一致性)？
6. 多语言公平性：同一段中文，字节级 BPE 的 token 数为什么比英文多？这对模型的“中文成本”意味着什么？

四、里程碑（可勾选，3-6 个）

- ☐ M1 手写 BPE 训练：能从语料统计词频、迭代合并、输出 merge 表与词表，并打印出前 10 个 merge 及其频次
- ☐ M2 手写编码器：支持特殊 token、未登录字节回退、往返一致性（自测 5 类边界：空串/单字节/中文/emoji/控制字符）
- ☐ M3 通过官方判分：test_tokenizer.py 与 test_train_bpe.py 全绿（原始输出贴 判卷记录.md）
- ☐ M4 压缩率对照：在 TinyStories (英) 与中文小语料上各训一个 32k 词表，出压缩率表 + 与你的下注对照
- ☐ M5 改坏实验（必做）：①禁用 bytes 回退 ②不处理特殊 token ③停止条件改成“合并到只剩 1 个词”——每个先写预测，再跑，再记录现象
- ☐ M6 一页报告 + 知识卡：报告按一页四段；知识卡 ≥ 3 张进 drill-ledger

五、交付物要求

交付物	位置	验收
实现代码	手写/（你写）	通过 M3
判分原始输出	判卷记录.md	粘贴 证据/judge-*.log 原文（不许转述）
证据	证据/	run.yaml（commit/时间/命令）+ 日志 + 压缩率表 + 改坏实验记录
报告	报告.md	一页四段：问题-动机-方法-结果
知识卡	memory/drill-ledger/topics/	≥3 张（bytes 回退 / merge 规则 / 压缩率权衡）

五之二、官方判分标准（2026-09-17 实跑探明，先知道标准再动手）

判分器已接通：环境/judge/judge.sh 01。实跑探明 **28 个测试**（27 项 + 1 项 xfail），要求如下：

接口契约（你唯一需要遵守的外部形状）

```
# 手写/bpe.py 必须提供：
def train_bpe(input_path, vocab_size, special_tokens, **kwargs) -> tuple[dict[int, bytes],
list[tuple[bytes, bytes]]]
class Tokenizer:
    def __init__(self, vocab, merges, special_tokens=None): ...
    def encode(self, text: str) -> list[int]: ...
    def decode(self, ids: list[int]) -> str: ...
    def encode_iterable(self, iterable) -> Iterator[int]: ... # ← 隐藏要求，见下
```

三条硬门槛（都会挂人）

#	测试	门槛	说明
1	test_train_bpe_speed	corpus.en + vocab 500 < 1.5 秒	官方参考 0.38s；“玩具实现”约 3s 会挂
2	test_train_bpe	merges 逐项 等于参考 merges；vocab 键值集合一致	意味着预处理与 并列打破规则 都要与参考一致
3	test_encode_iterable_memory_usage	用 <code>encode_iterable</code> 流式读 5MB 语料，额外内存 ≤ 1MB （RLIMIT_AS 实测）	必须有真正的流式实现 ，不能先 read() 全部

一个官方“故意让分”的测试（别被骗）

- test_encode_memory_usage 被官方标记 **xfail**，理由原话：“Tokenizer.encode is expected to take more memory than allotted (1MB)”。→ 也就是说：**encode() 超内存是允许的，encode_iterable() 超内存是不允许的**。这个 xfail 恰好是本题“接口设计决定内存上界”的活教材（见知识地图 M5 的同类问题）。

对齐口径的隐含要求

- 与 tiktoken 的 GPT-2 逐 id 对齐 → 编码时必须**严格按 merges 的 rank 顺序**合并，而不是“最长匹配优先”；且需处理 allowed_special 与重叠特殊 token（test_overlapping_special_tokens）。
- 往返一致性覆盖：空串 / 单字符 / Unicode / 多行 / 特殊 token 尾随换行 / 地址文本 / 德语 / TinyStories。

六、开题批（开场直接发给用户，3–5 题）

见 ../memory/HANDOFF.md 的“课题 01 开题批”五题（下注题 / 概念题 / 手算题 / 权衡题 / 工程题）。**第 1 题**（中文 token 倍数）必须先存档预测，它是本学科第一条“预测→打脸”链路。

七、讲课锚点（知行合一的“讲”这一腿）

- 讲什么：M2 的五个核心问题（压缩视角 → 字节起步 → 训练算法 → 编码与往返 → 多语言成本）
- 对标材料（三师制）：
 - 主线：CS336 A1 handout 的 tokenizer 章节（英文，规格权威）
 - 中文：李沐 d2l ch14.6 子词嵌入；或 minimind 的 `train_tokenizer.py` 中文注释（只读，不得抄进手写区）
 - 补充：HF LLM Course ch2（“Implement BPE/WordPiece/Unigram from the ground up”）
- 讲完必出代码级提问（3–5 道，示例）：
 1. 把 merge 循环里“统计相邻对频次”改成“只统计出现次数 >1 的对”，词表会变得更好还是更差？预测并说明。
 2. 你的 encode 如果按“最长匹配优先”而不是“按 merge 顺序”，往返一致性还成立吗？
 3. 词表里如果混入一个重复的 token，哪一步会先崩溃？

八、参考实现（只准用于对照，不准抄）

- karpathy/nanochat 的 nanochat/tokenizer.py（接口设计与特殊 token 处理）
- rasbt/LLMs-from-scratch ch2 附带的“BPE From Scratch”
- ⚠ 纪律：GS=看接口与设计思路，不看实现细节；你的代码必须自己写。判据：M5 的改坏实验能不能被你解释——抄来的人在这里一定卡住。

判卷记录 · 课题 01 BPE 分词器

判分器：环境/judge/judge.sh 01（CS336 官方 2026 测试，锁 commit a158843b2010，test_.py 未改一字）
 归档规则：只贴原始输出*，不做美化、不做转述（《判分与验收标准.md》§ 二-3）。

L1 · 基线（2026-09-17 立科时实跑）

项	结果
命令	环境/judge/judge.sh 01
结果	27 failed, 1 xfailed
失败原因	全部同一原因：FileNotFoundError：手写区还没有实现文件（接线层主动抛出的友好提示）
判分器状态	✓ 管线已验证可执行（28 个测试全部被收集并运行，非网络/环境错误）
结论	这是正确的初始状态：判分器就位，只差 手写/bpe.py

当初实跑的关键片段（完整输出会由脚本落到 证据/judge-*.log）：

```
FAILED tests/test_train_bpe.py::test_train_bpe_speed - FileNotFoundError
FAILED tests/test_train_bpe.py::test_train_bpe - FileNotFoundError
FAILED tests/test_train_bpe.py::test_train_bpe_special_tokens - FileNotFoundError
FAILED tests/test_tokenizer.py::... (24 项, 含 test_encode_iterable_memory_usage)
===== 27 failed, 1 xfailed in 3.67s =====
```

注：1 xfailed 不是缺陷——它是官方故意标记的 test_encode_memory_usage（理由：“encode 预期会超过 1MB 配额”）。详见 开题简报.md § 五之二。

L2 · 导师审稿意见

⊖ 待用户提交首次实现后填写（3-5 条：必考项 / 要记住 / 预告项）。

L3 · 费曼收口

☐ 未开始。收口动作：你对着“完全不懂分词的人”讲清 BPE，我装不懂连问三层（Why → Why not → What if），再抽 1 次“改坏实验”复述。

本课题的判分环境坑（已解决，记录备查）

坑	现象	处置
官方 conftest 需 Python ≥3.12	官方 pyproject.toml: requires-python = ">=3.12,<3.14"; conftest 用 PEP 695 泛型语法，3.11 无法解析	3.11 下自动改用等价轻量 conftest（同一 snapshot 夹具语义），并在日志头部如实声明；升到 3.12+ 后自动切回官方原文
tiktoken 需联网下载 GPT-2 词表	无外网时 SSLError 挂掉 12 项对齐测试	新增 环境/judge/prefetch_tiktoken.py: 用官方 fixture 离线构造 tiktoken 缓存(encoder.json 哈希与 tiktoken 内置期望一致，已验证 n_vocab=50257)

坑	现象	处置
PyTorch CPU 专用源不可达	download.pytorch.org SSL 被拦	bootstrap 改为：默认不装 torch（课题 01 不需要）；--with-torch 时先试 CPU 源、失败退回 PyPI

课题 02 开题简报 · Transformer 骨架与训练循环

模块: M0/M1/M3 | 周次: W2-W3 (09-21→10-04) | 算力: T0 纯 CPU, 0 GPU·小时 唐杰作业对应: ① 从零写 Transformer 的后半 | 判分来源: CS336 A1 tests/test_model.py、test_nn_utils.py、test_optimizer.py、test_serialization.py 手写: R2 (Transformer 全部组件) + R3 (训练循环与优化器) + R4 (反向传播验证)

一、研究背景

2017 年《Attention Is All You Need》删掉 RNN, 用自注意力 + 前馈层实现 **O(1) 依赖路径 + 全并行**。此后所有主流大模型都是它的变体, 但**细节已经换了一轮**: 正弦位置编码 → RoPE; LayerNorm → RMSNorm; post-norm → pre-norm; ReLU MLP → SwiGLU; MHA → GQA。本课题要求你把现代版写出来, 而不是复刻 2017 版。

二、研究问题（一句话）

一个“能从零训起来”的现代 Transformer 层, 最少需要哪几个组件? 每个组件去掉之后, 训练会以什么方式失败?

三、攻关线索

1. 形状先行: 写下每一步的张量形状 (含 batch、序列、头数), **先能在纸上跑通**, 再写代码。
2. 注意力的三个动作: 投影出 Q/K/V → 缩放点积 + 因果掩码 → softmax 加权求和并输出投影。哪一步最容易错? 为什么?
3. 位置信息: RoPE 为什么是“旋转 Q/K”而不是“加到输入上”? (讲义 M3 § 三给了恒等式)
4. 归一化与残差: 为什么 pre-norm 比 post-norm 好训? 残差提供了什么“通路”?
5. 优化器: AdamW 里“解耦权重衰减”解耦的是什么? 为什么它比 L2 正则更常用?
6. 稳定性: 梯度裁剪在什么情况下救命? fp16 vs bf16 的差别如何影响 loss 曲线?

四、里程碑

- ☐ M1 手写全部组件: 注意力 (含因果 mask 与 RoPE)、RMSNorm、SwiGLU FFN、残差堆叠、初始化
- ☐ M2 形状与数值判分通过: test_model.py / test_nn_utils.py 全绿
- ☐ M3 手写训练循环 + AdamW + LR 调度 + 梯度裁剪 → test_optimizer.py / test_serialization.py 全绿
- ☐ M4 端到端: CPU 上训一个极小模型 (d4-d8), loss 稳定下降, 并保存/加载 checkpoint 续训成功
- ☐ M5 R4 反向传播验证: 与 PyTorch autograd 的数值梯度对照 (相对误差 < 1e-6)
- ☐ M6 三组改坏实验 (去 mask / 换位置编码 / 去残差): 每组先写预测, 再跑, 再记录
- ☐ M7 一页报告 + 知识卡 ≥ 3 张

五、交付物

交付物	位置	验收
实现	手写/ (你写)	M2/M3 判分全绿

交付物	位置	验收
判分输出	判卷记录.md	证据/judge-*.log 原文
改坏实验记录	证据/改坏实验.md	预测→现象→解释 三段齐全
loss 曲线	证据/	训练日志 + 图（含坐标轴与步数）
报告	报告.md	一页四段

六、讲课锚点（讲义 M3 + 对标）

- **讲义**：讲义/M3-Transformer 内部机制.md（主干）+ 讲义/M0-数学与机器学习地基.md（§ 一 自动微分、§ 五 排查顺序）
- **对标**：CS336 lecture 03–04 · rasbt ch3–ch4 · 李沐 d2l ch10–ch11
- **必备工具链**：bash 环境/bootstrap_cpu.sh --with-torch（本课题起需要 torch）

七、代码级提问示例（讲课中出，先预测后验证）

1. 把因果 mask 从 softmax 之前挪到之后，训练现象会怎样？
2. 去掉残差后，4 层还能训、12 层完全不动——请用梯度路径解释。
3. 你把 RoPE 的最大频率底数从 10000 改成 100：短文训练会受影响吗？长上下文外推呢？
4. AdamW 的 weight_decay 若改成“直接加到梯度上”（即 L2），在大学习率下会出现什么差异？

八、风险与提醒

- ⚠ **CPU 训练极慢**：本课题只要能训起来、loss 下降即可，不追求指标（真训练在课题 04）。
- ⚠ **形状错误是最高频故障**：先写形状注释再写实现，可省一半时间。
- ⚠ **判分器需要 torch**；bootstrap --with-torch 首次约 555MB（沙箱网络受限时可能失败，届时改用 Kaggle CPU 会话）。

课题 03 开题简报 · Scaling Law 预演与打脸链路

模块：M4 | 周次：W4 (10-05→10-11) | 算力：T0 纯 CPU (极小规模 sweep) 唐杰作业对应：③ 拟合 scaling law 再外推 | 判分来源：无官方判分器 (CS336 A3 需 8 位学号 API key, 不可得) → 自建判分：预测 vs 实测的偏差 手写：R6 (拟合与外推)

一、研究背景

Kaplan 等 (2020) 与 Chinchilla (2022) 发现：loss 随参数量 N 与数据量 D 呈幂律下降，且存在一个算力最优配比。这条规律是“为什么敢花几千万训一次大模型”的全部依据。但初学者最常见的错误是：把外推当预言，不做回溯验证就把曲线画到天上去。

本课题的核心不是“拟合出一条漂亮的线”，而是建立“预测 → 验证 → 承认偏差”的科研习惯。这也是本学科“预测—打脸日志” (memory/MEMORY.md) 的主要产地。

二、研究问题（一句话）

在极小规模（我的 CPU 能承受的范围内）跑 4–5 个点，能否外推出更大模型的 loss？外推误差有多大、什么时候会崩？

三、攻关线索

- 口径：横轴用什么？参数量 N ？token 数 D ？（提示：固定预算时， N 与 D 不能同时变）
- 记账：用 M0 的 $6ND$ 估算每一次跑的算力，确保不同点是在“同样算力预算”下比较（否则曲线无意义）
- 拟合：幂律 $L = aN^{-\alpha} + c$ 的形式里，那个常数项 c 代表什么？它会不会让外推失效？
- 外推：拟合完先把预测写死在文件里（带日期），再去跑验证点 —— 顺序不能反
- 偏差来源：小规模数据是否“无限数据区”？学习率是否随规模重新调过？这些都是外推失准的技术原因

四、里程碑

- M1 设计 sweep：4–5 个规模点（如 d4/d6/d8/d10），预算固定、口径统一
- M2 跑完 sweep，记录每个点的最终 loss/bpb 与耗时（CPU 上即可，分钟级）
- M3 拟合幂律并画图（含数据点、拟合线、坐标轴标注）
- M4 先写死外推预测（例如 d12 的 bpb 预测值 + 置信区间），归档到 证据/
- M5 跑验证点，计算偏差，并解释偏差的方向与原因
- M6 一页报告：包含“预测 vs 实测”对照表（这一条是验收硬指标）

五、交付物

交付物	位置	验收
sweep 原始数据	证据/sweep.csv	含配置、loss、耗时、算力估算
拟合脚本与图	脚手架/ + 证据/	图有坐标轴、有拟合优度
外推预测存档	证据/预测-<日期>.md	跑验证点之前的文件时间戳
报告	报告.md	预测 vs 实测对照表

六、讲课锚点

- 讲义：讲义/M4-M9-待展开大纲.md 的 M4 部分 (W3 展开为完整版 M4-训练.md)

- **对标**: CS336 lecture 09/11 (scaling laws) · nanochat/runs/scaling_laws.sh 与其 miniseries 文档 · d2l 的线性回归/最小二乘 (拟合的数学基础)
- **搭桥**: probability/ 的方差与回归 —— 拟合优度、残差分析直接可用

七、代码级提问示例

1. 为什么横轴常画 **FLOPs** 而不是参数量? (提示: Chinchilla 的核心结论是什么)
2. 你拟合出的 c (常数项) 为负, 说明了什么? 还能外推吗?
3. 如果两个规模点用了不同的学习率, 曲线还能一起拟合吗? 为什么?
4. 【下注】 你预测 d12 的 bpb 会比你拟合线的预测值偏高还是偏低? 先写下答案再跑。

八、风险与提醒

- ⚠ **不要为了“好看”调学习率**: 每个点的超参策略必须写进 证据/, 否则外推无意义。
- ⚠ 极小规模会出现“学习率不敏感/过于敏感”的异常区间 —— 这本身就是很好的报告素材 (no-go 不擦除)。
- ⚠ 本课题**不花 GPU**; 如发现 CPU 太慢, 优先“缩小语料/减步数”, 不要上 Kaggle (配额留给课题 04)。

课题 04 开题简报 · 0.1B 端到端预训练（首次上机）

模块: M4/M5 | 周次: W5-W6 (10-12→10-25) | 算力: T2 Kaggle 2×T4, 计划 12-21 GPU·小时 唐杰作业对应: ① 端到端训一个 0.1B | 判分来源: 自建 (loss/bpb 达标 + 采样可读性) + nanochat/core_eval.py 的 CORE 手写: 复用课题 01/02 的产物 (tokenizer + 模型 + 优化器), 本课题新增数据管线与训练编排的判断

一、研究背景

“0.1B”是唐杰作业里的数字,也是单卡/双卡免费算力下的上限。参考点: nanochat 的 d12 约等于 GPT-1 量级 (官方讨论区原话: “d12 是我最喜欢的模型,它是 GPT-1 的大小,约 6 分钟训完”——那是在 8×H100 上)。本课题的核心不是刷指标,而是把“训练一次完整模型”的每个环节亲自走通并记账。

二、研究问题（一句话）

在 2×T4、20 GPU·小时的预算内,我能不能从零训出一个“能生成连贯英文、并在 CORE 上有非零分”的 0.1B 模型? 实测成本与估算差多少?

三、攻关线索

- 1. 数据: TinyStories (全量) + owt-sample 采样。先算一遍: 词表大小 × 语料字节 → token 数 → 需要的步数
- 2. 规模选择: nanochat 用单一旋钮 --depth; 在 T4 上 16GB 显存能放多深? (用 M0 的显存公式先估)
- 3. 精度: T4 是 SM75, 没有 bf16 → 必须 fp16 + GradScaler (或用 nanochat 的 NANOCHAT_DTYPE=float16)
- 4. 断点续训: Kaggle 会话 9-12h 会断 → checkpoint 必须写 /kaggle/working (20GB 持久)
- 5. 超时自停: 脚本内置 wall-clock 上限, 防止忘记关会话
- 6. 指标: 训练看 bpb (词表无关), 最终跑 CORE (22 数据集合成, core_eval.py 单文件)

四、里程碑

- ☐ M1 实验卡获批: 写清假设/命令/预算上限/停止条件/成功判据 (见《算力与成本预算.md》§ 五)
- ☐ M2 会话预检: nvidia-smi 确认 2×T4 (若是 P100 立即重开会话)
- ☐ M3 冒烟: --depth=4、100 步, 确认能跑、能存、能续训 (≤10 分钟)
- ☐ M4 首次正式训练: 双 T4 + DDP, 跑到预算上限或 loss 平台
- ☐ M5 续训一次 (验证 checkpoint 真的可用 —— 这一条最容易翻车)
- ☐ M6 评测: bpb + CORE + 采样 20 条样本 (人工判断“是否连贯”)
- ☐ M7 成本账单: 实测耗时 vs 估算 (M0 的 6ND + T4 折算), 逐项解释偏差
- ☐ M8 一页报告 + 知识卡 (含“我最意外的三个现象”)

五、交付物

交付物	位置	验收
实验卡	证据/实验卡-NN.md	经用户确认
run.yaml + 日志	证据/	含 commit、配置、随机种子、耗时
曲线	证据/	loss/bpb vs FLOPs 与 vs 时间 两张
采样样本	证据/样本.md	≥20 条, 标注“通顺/不通顺”

交付物	位置	验收
成本账单	报告.md	实测 vs 估算偏差 $\leq 2\times$ 或给出解释
checkpoint	Kaggle 持久盘	不进程（体积原因）；只在 证据/ 留哈希与路径

六、讲课锚点

- 讲义：M4-训练.md（W3 展开）
- 对标：nanochat/README.md 与 runs/speedrun.sh（读它怎么组织全流程）· CS336 lecture 05/13-14
- 搭桥：课题 03 的 scaling 结论在这里第一次被“真跑”检验

七、代码级提问示例

1. 双 T4 跑 d12，为何不是 2 倍加速？先预测扩展效率（50%？80%？），再测。
2. T4 上 fp16 与 fp32，谁更快？谁更稳？GradScaler 在解决什么？
3. 你的 loss 在第 2000 步开始上升。按 M0 § 五 的顺序，你的前三个动作是什么？
4. 若把语料从 TinyStories 换成中文语料（词表不变），loss 会怎么变？bpb 呢？

八、风险与提醒

- **！ P100 陷阱**：本课题如果拿到 P100，训练慢 3-5 倍且后续课题 05 直接不可用 → 宁可等 T4 时段。
- **！ 配额**：本课题占全学期约 1/5 的配额，实验卡必须写实；冒烟不通过不许开正式训练。
- ⚠ 0.1B 在 T4 上大概率需要“降 batch + 梯度累积”，这会改变有效 batch size → 要写进 run.yaml。
- ⚠ Kaggle 会话到期会杀进程：每 500 步存一次是硬要求。

课题 05 开题简报 · 手写 Triton attention kernel

模块：M5 | 周次：W7-W8 (10-26→11-08) | 算力：T1 Kaggle 1×T4，计划 10 GPU·小时 唐杰作业对应：② 手写 Triton attention kernel，测多卡训练与推理增益 | 判分来源：CS336 A2（正确性对齐）+ 自建性能对照 手写：R5（kernel 本体：分块 + online softmax）

一、研究背景

朴素注意力要把 $L \times L$ 的注意力矩阵整个写进显存： $L = 8192$ 时单头就是 64M 个元素。FlashAttention (2022) 的核心洞察：**根本不需要把矩阵写出来**——分块计算、在线维护 softmax 的最大值与归一化因子，就能得到数学上完全等价的结果。它把注意力的显存从 $O(L^2)$ 降到 $O(L)$ 。

Triton 让这件事对非 CUDA 专家也可做：用 Python 风格的 DSL 写“块级”程序。本课题的目标不是写出最快的 kernel，而是亲手经历“数值等价 + 显存下降 + 速度提升”这三件事的因果链。

二、研究问题（一句话）

一个分块 + online softmax 的手写 kernel，能否在数值上等价于朴素实现，并把显存从 $O(L^2)$ 降到 $O(L)$ 、速度提升若干倍？提升来自哪里（算术减少？访存减少？）

三、攻关线索

1. 先量化瓶颈：朴素实现在 $L=2k/8k$ 时的显存、耗时（用 profiler 看是 compute-bound 还是 memory-bound）
2. online softmax 的递推：维护 running max m 与 running sum ℓ ，新块的贡献要按 $e^{m_{\text{old}} - m_{\text{new}}}$ 缩放——这正是 M0 §四 logsumexp 的应用
3. 分块维度：按 query 分块、沿 key 方向扫描 → 每个块只需常驻 SRAM
4. 数值容差：与 PyTorch SDPA 对照，相对误差应在 $1e-2 \sim 1e-3$ (fp16)；先写容差，再判定
5. 多卡/推理增益：训练侧测吞吐 (tokens/s)，推理侧测 decode 显存与延迟——这是唐杰作业②要求的“增益”

四、里程碑

- □ M1 基线：朴素注意力的显存与耗时 ($L=2k/4k/8k$ 三点)
- □ M2 正确性：手写 kernel 与 SDPA 的数值对照 (max abs error 记录在案)
- □ M3 显存：证明峰值显存随 L 近似线性（对比朴素实现的 $O(L^2)$ ）
- □ M4 速度：给出 speedup 曲线（含 L 与 block size 两个维度）
- □ M5 profiling：用 profiler 说明“提升来自哪里”（访存 vs 算术，给出数字）
- □ M6 增益测量：训练吞吐 (tokens/s) 与推理（融合进 attention 后）的实测对比
- □ M7 一页报告 + 知识卡（online softmax 递推必须能徒手推）

五、交付物

交付物	位置	验收
kernel	手写/（你写）	M2 数值对照通过
三张图	证据/	speedup / 显存 / profile（坐标轴与硬件口径必须标注）

交付物	位置	验收
数值误差记录	证据/	含容差设定与依据
报告	报告.md	必须回答“提升来自哪里”

六、讲课锚点

- 讲义：讲义/M5-推理.md (W5 展开) 中“GPU 内存层级与 Triton 编程模型”一节
- 对标：CS336 lecture 05-06 (GPUs / Kernels & Triton) + A2 handout (**flash-attn 需两步安装**：
uv sync --no-install-package flash-attn 再 uv sync)· Triton 官方 FlashAttention 教程
- 搭桥：M0 § 四的 logsumexp —— 你会亲手把它写进 kernel

七、代码级提问示例

1. online softmax 里，为什么新块要乘 $e^{m_{old}-m_{new}}$ ？不乘会怎样？
2. block size 从 64 增到 256：显存、速度、数值误差各往哪走？
3. 为什么 L 很大时，朴素实现容易 OOM 而 kernel 不会？(用 $O(L^2)$ vs $O(L)$ 说清)
4. 【下注】你预测 speedup 在 L=2k 时是多少倍？先写，再测。

八、风险与提醒

- **！ P100 不可用于本课题**：Triton 要求计算能力 ≥ 7.0 ，P100 是 6.0 $\rightarrow$ 开跑前必须验明是 T4。
- ⚠ 正确性优先于速度：**数值对照不过，速度数字一律不算数**。
- ⚠ 硬件口径陷阱：任何 speedup 数字必须写明“对比对象 (PyTorch SDPA / 朴素实现)”与“硬件 (T4)”，否则曲线毫无意义（这一条写进报告硬要求）。
- ⚠ 若 T4 上 kernel 因共享内存不够而失败，**先降 block size，不要放弃**（这本身就是教学素材）。

课题 06 开题简报 · 多卡与分布式训练

模块: M4 | 周次: W9 (11-09→11-15) | 算力: T2 Kaggle 2×T4, 计划 8 GPU·小时 唐杰作业对应: ② 的“多卡训练增益”部分 | 判分来源: CS336 A2 分布式部分 + 自建 benchmark 手写: R3 的延伸 (并行策略选择与通信原语判断)

一、研究背景

数据并行 (DDP) 把同一模型复制到每张卡、各自算一个微批、再**同步梯度**; 当模型放不下单卡时, 才有 FSDP/ZeRO (切分参数、梯度、优化器状态)、张量并行 (切层内矩阵)、流水线并行 (切层)。Kaggle 免费档给的是 2×T4, 且走 PCIe 而非 NVLink —— 这恰好是“观察通信成为瓶颈”的最佳实验台。

二、研究问题

在弱互联 (PCIe) 的 2×T4 上, DDP 的扩展效率是多少? 瓶颈是计算还是通信? FSDP 在什么规模才开始划算?

三、攻关线索

- 1. 先对齐正确性: DDP 后的梯度应与单卡 (同等效批大小) **数值一致**, 先验证再谈性能
- 2. 批大小语义: 2 卡 × 每卡 batch = 等效批大小; 若显存受限要**同步调整学习率** (线性/平方根缩放)
- 3. 计时口径: step time 要区分“纯计算时间”与“含 all-reduce 的时间”; 用 torch.profiler 或手写计时
- 4. 扩展效率: $E = \frac{T_1}{2T_2}$ 。先**预测** (PCIe 下 0.6–0.8?) 再测
- 5. 通信占比: 梯度同步量与参数量成正比 (2N 字节/步, all-reduce 近似); 序列长/批小时通信占比更高
- 6. FSDP 的代价: 切分带来额外通信 (all-gather + reduce-scatter), **小模型上往往更慢**

四、里程碑

- ☐ M1 DDP 梯度一致性验证 (与单卡同等效批) → 记录最大误差
- ☐ M2 单卡 vs 双卡的 step time、tokens/s 对照 (同模型、同等效批)
- ☐ M3 **扩展效率曲线** (至少 3 个配置点: 不同 batch/序列长度)
- ☐ M4 **通信占比分解**: 计算 / all-reduce / 数据加载 三段计时
- ☐ M5 一次 FSDP 实验并解释“为什么 (不) 划算”
- ☐ M6 **先写预测再实测**: 预测扩展效率, 跑完对照 (进 memory/MEMORY.md 打脸日志)
- ☐ M7 一页报告

五、交付物

交付物	位置	验收
分布式脚本	手写 / 或 脚手架 / (按 B3 分类)	可复跑
计时原始数据	证据/bench.csv	含每配置 3 次重复
扩展效率曲线	证据/	标注硬件 (2×T4 PCIe) 与对比基准
报告	报告.md	解释“为什么不是 2 倍”

六、讲课锚点

- 讲义: M4-训练.md (W3 展开) 中并行一节

- **对标**: CS336 lecture 07-08 (Parallelism) · nanochat/optim.py (分布式优化器实现) · HF Ultra-Scale Playbook (用前先核)
- **搭桥**: M0 的“梯度累加” → DDP 就是跨卡的梯度累加

七、代码级提问示例

1. $2 \times T4$ 上 DDP 只快了 1.4 倍，列出三个最可能的原因。
2. 把每卡 batch 减半、用梯度累积补回等效批，通信量会怎么变？
3. 什么情况下 FSDP 会比 DDP 慢？（用通信量解释）
4. all-reduce 的通信量是多少字节/步？用参数量 N 表示。

八、风险与提醒

- ⚠ Kaggle 双 T4 会话**不稳定**（可能只给一张）：预检时确认 `torch.cuda.device_count()==2`。
- ⚠ 不要在本课题纠缠“调到最快”——目标是理解扩展效率的成因，配额要留给课题 08。

课题 07 开题简报 · 数据清洗与去重

模块：M7/M4 | 周次：W10 (11-16→11-22) | 算力：T1 Kaggle 1×T4，计划 6 GPU·小时 唐杰作业对应：③ 从 raw dump 洗语料 | 判分来源：CS336 A4 (过滤/去重规格) + 自建消融 手写：R8 (过滤/去重的判定规则设计)

一、研究背景

“数据决定上限”已是共识：C4/FineWeb/Dolma 等工作的主要贡献不是模型而是数据管线。典型手段：语言识别、质量启发式（重复度、符号比、平均行长）、有毒内容过滤、近似去重（MinHash/LSH）。最有教学价值的现象：**清洗规则过严会伤害模型**（把有价值的长尾数据也删了）。

二、研究问题

在固定算力下，“清洗 + 去重”能把同样规模模型的下游表现提升多少？每条规则的收益与代价各是多少？

三、攻关线索

1. **基线优先**：先训一个“未清洗”的基线（可与课题 04 的结果复用）—— **没有对照就无法声称“清洗有效”**
2. **规则设计**：至少三条可解释规则（如：短行比例、符号-词比、n-gram 重复率），每条要能说清“它在拦什么”
3. **去重量级**：精确去重（哈希）与近似去重（MinHash + LSH）的成本差异；文档级 vs 段落级
4. **消融**：逐条关掉规则各跑一次（小规模即可），得“每条规则的边际贡献”表
5. **口径**：所有对照必须同 **token 预算、同分词器、同评测**（否则结论无效）
6. **常用工具对照**：datajuicer/data-juicer (Apache-2.0, 活跃) 与 allenai/dolma 的规则集可作参考

四、里程碑

- ☐ M1 数据来源与规模确定（用 owt-sample / TinyStories 扩样，或小规模 CC 采样）
- ☐ M2 手写 3 条启发式规则 + 1 个去重算法（精确哈希起步，有余力再上 MinHash）
- ☐ M3 规则单元测试（含**假阳性样例**：被误删的好数据长什么样）
- ☐ M4 训练对照：未清洗 vs 全部清洗 vs 逐条消融（≥4 组跑）
- ☐ M5 结果表：每组的下游指标 + **数据保留率**（这是被忽略的关键列）
- ☐ M6 一页报告：必须回答“哪条规则最值钱、哪条可能有害”

五、交付物

交付物	位置	验收
过滤/去重实现	手写/	M3 单测通过
规则清单与理由	报告.md	每条规则说明“拦什么、代价是什么”
对照实验数据	证据/对照.csv	含保留率与指标
假阳性样例	证据/	≥5 条被误删的好数据

六、讲课锚点

- **讲义**：M7-评测.md (W9 展开) 中“数据质量与评测的关系”
- **对标**：CS336 lecture 13-14 (Data) + A4 handout · datajuicer/data-juicer 规则文档 · Dolma 论文
- **搭桥**：概率论的采样与分布 —— 过滤本质是“改变训练分布”，会引入选择偏差

七、代码级提问示例

1. 过滤掉“短行”能提分，但会不会系统性删掉对话/代码类数据？怎么检测这种偏差？
2. 精确去重 vs 近似去重：各自漏掉什么、误杀什么？
3. 数据保留率从 90% 降到 50%，你如何判断“这是提纯还是阉割”？
4. 【下注】预测“全部清洗”相对“未清洗”的提升幅度，先写再测。

八、风险与提醒

- ⚠ 磁盘：本课题可能涉及较大语料，**大文件绝不进 git**（见学科 .gitignore）。
- ⚠ 不要一开始就上 MinHash 集群版：**先精确去重跑通闭环**，再考虑近似。
- ⚠ 4 组对照训练会吃掉配额：**用小模型 + 短序列**，保证口径一致即可。

课题 08 开题简报 · SFT / DPO / RLVR 三方对照

模块: M6/M7 | 周次: W11-W12 (11-23→12-06) | 算力: T2 Kaggle 2×T4, 计划 18 GPU·小时 (全学期最贵) 唐杰作业对应: ④ 同一基座上把 SFT、DPO、RLVR 做对照 | 判分来源: CS336 A5 tests/test_grpo.py + 自建三方对照 手写: R7 (损失函数与优势估计)

一、研究背景

预训练给“知识”，后训练给“行为”。三条主流路线：

方法	需要什么	优化什么
SFT	(指令, 回答) 对	最大似然 (只对回答部分算 loss)
DPO	(指令, 好回答, 坏回答) 偏好对	隐式奖励差 (无需显式 reward model)
RLVR	可判对错的答案 + 验证器	期望可验证奖励 (GRPO 用组内相对优势)

本课题的教学价值极高：它是课程大纲“机器学习（强化学习）”与唐杰作业④的交点，也是“知识讲解”最能发挥的地方——GRPO 的每一项都能在概率论里找到根（期望、方差、条件期望）。

二、研究问题

同一个基座、同一套评测，SFT / DPO / RLVR 各自把什么能力提升了、把什么能力损害了？训练奖励的上升，有多少是真的能力提升？

三、攻关线索

- 基座与数据**：基座用课题 04 的模型（或 0.1B 级开源小模型）；任务是可验证的（数学题/格式约束/单元测试）
- SFT 的损失掩码**：只对回答部分算 loss，指令部分不算——这是个高频错误点
- DPO 的推导**：为什么 $\sigma\left(\beta \log \frac{\pi(y_w)}{\pi_{ref}(y_w)} - \beta \log \frac{\pi(y_l)}{\pi_{ref}(y_l)}\right)$ 能等价于“隐式奖励”？
- GRPO 的四步**：采样一组输出 G 个 → 打分 → 组内归一化得优势 $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ → 加权策略梯度 + KL 惩罚
- KL 惩罚的作用**：防止策略跑离参考模型太远（对应 RLHF Book 的“过度优化”章）
- 评测口径**：训练奖励 ≠ 能力。必须在 held-out 题集上评，并且给误差棒
- 制造一次 reward hacking**：例如让模型学到“输出格式对了但答案是瞎猜”，记录现象（这是本课题最好的素材）

四、里程碑

- M1 SFT 基线（含损失掩码正确性验证：打印一条样本的 mask）
- M2 DPO 跑通 + 与 SFT 的对照（held-out）
- M3 手写 GRPO 损失并逐项注释（优势从哪来、为什么要除标准差）
- M4 tests/test_grpo.py 通过（判分器接入后）
- M5 三方对照表（同基座、同评测、同种子策略，带误差棒）
- M6 reward hacking 实验：故意设计一个“可被钻空子”的奖励，观察并记录
- M7 一页报告 + 知识卡（GRPO 递推必须能徒手推）

五、交付物

交付物	位置	验收
三个训练脚本	手写/（损失与优势估计属 R7）	可复跑
判分输出	判卷记录.md	test_grpo.py 原文
对照表	证据/	含误差棒与评测口径说明
reward hacking 记录	证据/	现象 + 机制解释
报告	报告.md	“能力提升 vs 训练奖励” 必须分开讨论

六、讲课锚点

- 讲义：M6-后训练.md（W10 展开）
- 对标：rasbt《Reasoning from Scratch》ch3（先建评测）+ ch6–ch7（GRPO）·《RLHF Book》ch4/6/8/14·CS336 A5 + lecture 15–17·minimind/trainer/train_grpo.py（中文工程实现）·nano-aha-moment（单文件）
- 搭桥（重点）：probability/ 的期望、方差、条件期望——GRPO 的优势归一化就是“标准化”，KL 惩罚就是“分布差异”，REINFORCE 就是“用样本均值估期望”

七、代码级提问示例

1. 为什么 SFT 只对回答部分算 loss？若指令部分也算，会发生什么？
2. GRPO 里为什么要除以组内标准差？如果一组所有奖励完全相同，优势是多少？这时梯度会怎样？
3. DPO 不需要 reward model，那它的“奖励”藏在哪儿？（用对数比写出来）
4. 训练奖励从 0.3 涨到 0.8，held-out 准确率却从 40% 掉到 35%。列出三个可能原因。
5. 【下注】预测三方在 held-out 上的排序，先写再测。

八、风险与提醒

- **！最贵的一周**：18 GPU·小时是全学期 1/5，三个方法各只跑一次正式实验，其余用小规模调试。
- **！不做 held-out 评测就等于自欺**：训练奖励必须与独立评测同时报（写进报告硬要求）。
- ⚠ A5 官方仓库未声明 SPDX 许可 → 只做本地学习对照，不复制进仓（见《上游锁定清单》§ 三）。
- ⚠ Kaggle 会话上限 9–12h：RL 训练务必支持断点续训，否则一次会话跑不完。

课题 09 开题简报 · 可验证环境 + harness

模块: M7/M8 | 周次: W13 (12-07→12-13) | 算力: T2 Kaggle 2×T4, 计划 6 GPU·小时 唐杰作业对应: ⑤ 搭可验证环境 + harness | 判分来源: 自建 (判定逻辑单测 + rollout 成功率) 手写: R9 (判定逻辑 / verifier)

一、研究背景

2026 年的关键词是“可验证奖励”: 与其训练一个会拍马屁的奖励模型, 不如造一个能判定对错的环境。CS336 没有这一环, Berkeley Agentic AI MOOC 专门有一讲叫 “Post-Training Verifiable Agents”, PrimeIntellect-ai/verifiers 把它抽象成库。唐杰作业⑤要求“搭可验证环境 + harness”正是这个方向。“可验证”的三要素: ①任务是可判定的 (不是主观好坏) ②环境能给出稳定反馈 ③harness 能重复运行并复现。

二、研究问题

我能为一个小程序设计一个“可自动判定”的任务环境吗? 判定逻辑的假阳性/假阴性有多高? 模型在这个环境里的成功率是多少?

三、攻关线索

- 1. 选题: 从三个里选一个 (按成本从低到高)
 - ①数学题解答 (用数值比对 + 容错; MATH-500 风格)
 - ②函数补全 (隐藏单元测试; 本质是迷你 HumanEval)
 - ③格式约束生成 (如严格 JSON / 表格填充, 规则判定)
- 2. 判定逻辑的三大坑: 等价但不相同 ($1/2$ vs 0.5)、解析失败当错、边界用例漏判
- 3. 假阳性检测: 人为构造“错误但能骗过判定器”的答案 → 测出假阳性率 (这是质量的核心指标)
- 4. harness 设计: 任务装载 → 模型生成 → 判定 → 记录轨迹 → 复现 (固定种子)
- 5. 可重复性: 同一批任务跑两次, 成功率波动多大? (误差棒思维, M7 讲过)
- 6. 对照: 可验证奖励 vs 规则弱判定 (如只看格式), 用同一模型比成功率差异

四、里程碑

- ☐ M1 选定任务类型, 写 30–100 道题的题集 (含标准答案与判定规则)
- ☐ M2 判定逻辑单测: 等价形式、边界、异常输入 (≥ 10 个用例)
- ☐ M3 假阳性实验: 构造 10 个“错误但可能蒙混”的答案, 测出漏判率
- ☐ M4 harness 跑通: 给基座模型跑一遍, 得成功率 + 失败样本
- ☐ M5 失败分类: 把失败归成 3–5 类 (格式错 / 计算错 / 中途放弃 / 判定器误杀)
- ☐ M6 一页报告: 判定器质量 + 模型表现 两个都要报

五、交付物

交付物	位置	验收
题集与判定器	手写 / (判定逻辑 R9)	M2 单测通过
harness 脚本	脚手架 / (B3)	固定种子可复跑
失败样本	证据 /	≥ 20 条, 分类标注

交付物	位置	验收
假阳性报告	报告.md	含漏判率与典型案例

六、讲课锚点

- 讲义：M8-智能体.md（W13 展开）
- 对标：Berkeley Agentic AI MOOC 的“Post-Training Verifiable Agents”与“Agent Evaluation”讲 · PrimeIntellect-ai/verifiers（只读概念，不引入代码）· harbor-framework/terminal-bench 的任务设计思路 · rasbt ch3（评测器设计：解析 + verifier + 基准式检查）

七、代码级提问示例

1. 你的判定器把 1/2 判错、把 0.50 判对——这属于哪一类错误？如何系统性避免？
2. 假阳性率高会导致 RL 训练出现什么后果？（联系课题 08 的 reward hacking）
3. 同一批任务两次运行成功率差 8%。你会先怀疑模型还是 harness？怎么验证？
4. 为什么“用 LLM 当裁判”在可验证任务上通常是退步？

八、风险与提醒

- ⚠ 判定器本身也要被评测：只报模型成功率不报判定器质量，是不完整的实验。
- ⚠ 题集不要抄公开 benchmark（污染问题），自己出题 + 自己判定才是本课题的价值。
- ⚠ 配额有限：优先“题集质量高、任务简单”，不要一上来做代码执行沙箱（那是课题 10 的事）。

课题 10 开题简报 · 长程 Agent 与 self-judge

模块: M8 | 周次: W14 (12-14→12-20) | 算力: T2 Kaggle 2×T4, 计划 8 GPU·小时 唐杰作业对应: ⑤
训长程 Agent, 鼓励 self-judge loop | 判分来源: 自建 (消融实验 + 轨迹日志) 手写: R10 (self-judge 协议设计)

一、研究背景

从“回答问题”到“完成任务”的跨越, 靠的是 **harness (外壳)**: 工具调用、上下文管理、失败重试、自我评估。Karpathy 与李宏毅都专讲 **context engineering**; nanochat 里 `execution.py` 就是一个最小的“能执行代码的 Agent 外壳”。本课题的落点是**长程任务**: 多步串联后误差累积, 以及 **self-judge (自我评判)** 到底有没有用。

二、研究问题

多步任务中, 失败主要来自模型能力还是来自 harness 设计? 加上 self-judge loop 后成功率提升多少——还是反而下降?

三、攻关线索

1. 最小 ReAct: 不用任何框架, 手写 Thought → Action → Observation 循环 (三四十行就能跑)
2. 工具设计: 先给两个工具 (计算器 / 受限代码执行)。工具接口的清晰度往往比模型智力更决定成败
3. 长程任务构造: 把课题 09 的题改造成“需要 3-5 步”的复合任务 (如: 先算 → 再验 → 再格式化)
4. 失败分类: 格式错 / 工具调错 / 目标漂移 (忘了原始要求) / 循环打转 / 提前放弃
5. self-judge 的两种形态:
 - 自评重试: 模型给自己打分, 低于阈值就重做 (rasbt ch5 的 self-refinement)
 - 自评过滤: 生成多个候选, 自己挑最好的 (best-of-N 的变体)
6. 关键实验 (消融): 同样的任务集, 三组对照 = 无 self-judge / 自评重试 / 自评过滤
7. 警惕: self-judge 会放大原有错误 (模型判不出自己错在哪, 反而确信) → 必须留失败案例

四、里程碑

- ☐ M1 手写最小 ReAct 循环 (无框架), 跑通单步任务
- ☐ M2 加入第二个工具 + 多步任务集 (≥30 个任务)
- ☐ M3 轨迹记录规范: 每步的 thought/action/observation 落盘 (可复现)
- ☐ M4 基线成功率 + 失败五分类 (每类给 2 个真实案例)
- ☐ M5 self-judge 消融: 三组对照, 给出成功率差与误差棒
- ☐ M6 反例收集: 找出 ≥3 个“self-judge 导致更差”的案例并解释机制
- ☐ M7 一页报告 + 知识卡

五、交付物

交付物	位置	验收
ReAct 循环	手写/	可跑通多步任务
self-judge 协议	手写/ (R10)	有明确的判定时机与阈值设计说明
轨迹日志	证据/trajectories/	每任务一份, 含步数与结果

交付物	位置	验收
消融数据	证据/	三组成功率 + 误差棒
报告	报告.md	必须回答“何时 self-judge 有害”

六、讲课锚点

- 讲义：M8-智能体.md (W13 展开)
- 对标：李宏毅《Context Engineering》专讲 (Agent 时代的关键技术) · Berkeley LLM Agents MOOC (ReAct 原始论文那讲) · HF Agents Course (三框架对照 + 观测性/评测 bonus unit) · **rasbt ch5 (self-refinement)** · nanochat/execution.py+tests/test_execution.py (**沙箱执行的最小实现**, 可读接口不可抄实现)
- 搭桥：M5 的 KV cache (长上下文的显存代价) → 为什么长程 Agent 必须做上下文压缩

七、代码级提问示例

1. 长程任务失败率随步数指数上升。**分解**：是每步错误率问题，还是错误会累积传播？
2. self-judge 什么时候**一定**无效？(提示：当模型无法区分“看起来对”和“真的对”时)
3. 上下文压缩 (截断/摘要) 会引入什么新失败模式？
4. 为什么“工具返回值要结构化”能显著提升成功率？(从 token 分布角度解释)
5. 【下注】预测 self-judge 在简单任务 vs 困难任务上的效果差异，先写再测。

八、风险与提醒

- **！ 不要用框架起步**：手写一遍 ReAct 再去看 LangGraph/smolagents, 否则学不到 harness 的本质。
- ⚠ self-judge 会显著增加 token 消耗与时间 → 算力预算要写进实验卡。
- ⚠ 轨迹日志可能很大 → 只保留结构化摘要 (thought 截断 + 结果), 避免污染仓库体积。

课题 11 开题简报 · 结项 (NeurIPS 报告 + 现场 demo)

模块: M9 | 周次: W15-W16 (12-21→2027-01-03) | 算力: 弹性 ≤ 4 GPU·小时 唐杰作业对应: ⑥ 英文 NeurIPS 格式 + W16 现场 demo、⑦ 作业 40% / 大项目 60% | 判分来源: 报告协议 + 导师审稿 + 自我攻击

一、研究背景

唐杰的原话是“问题、动机、方法、结果，一项都不能糊”。这恰好也是本学科九份章程反复强调的：没有证据的结论不采纳、no-go 不擦除、误差棒必填。结项不是“写一篇总结”，而是把 11 个课题的证据链组装成一份可被审稿的作品。

二、研究问题 (结项版)

我能不能用一份英文技术报告 + 一个可运行的 demo，证明我独立完成了大模型全链路，并且知道自己的结论在哪里不成立？

三、攻关线索

1. 报告骨架 (NeurIPS 风格, 8-10 页): Abstract / Introduction / Related Work / Method / Experiments / Results / Discussion & Limitations / Conclusion
2. 只写有证据的结论: 每个数字都能指回 证据/ 中的原始日志
3. Limitations 必须诚实: 规模受限 (0.1B vs 前沿)、算力受限 (免费档)、评测口径的局限
4. Demo 设计: 现场演示要可在 3 分钟内跑完 (预生成 checkpoint, 不现场训练)
5. 自我攻击 3 条: 主动写出“我的结论最可能错在哪”
6. 一稿多用: 这份报告同时作为课程大论文 (70%) 与唐杰作业⑦的交付物

四、里程碑

- ☐ M1 证据清点: 11 个课题的 证据/ 全部齐备, 缺的补齐 (这是一次“审计”)
- ☐ M2 英文报告初稿 (Abstract + Method + Experiments 先写)
- ☐ M3 图表规范化: 所有图有坐标轴、单位、误差棒、样本数
- ☐ M4 自我攻击 3 条 + 导师审稿 3-5 条 → 修订
- ☐ M5 Demo 准备: 脚本 + 预生成模型 + 3 分钟演练
- ☐ M6 总复盘: 写进 memory/LEARNINGS.md (哪些讲法有效、哪些坑最贵)
- ☐ M7 课程大论文定稿并提交 (署名直填: 孙承泽 / 2253710052 / 能动强基 2501)

五、交付物

交付物	位置	验收
英文报告	课题 11-结项/report-en.md (+ PDF)	格式合规、结论有据
中文大论文	课程轨/大论文/	课程要求格式
Demo	课题11-结项/脚手架/demo.sh	3 分钟内可跑
复盘	memory/LEARNINGS.md + HANDOFF.md	可被下一任直接接续

六、讲课锚点

- 讲义: M9-前沿.md (W15 展开)

- **对标:** rasbt ch3 的“先建评测”精神贯穿全文 · CS336 报告规范 · 论文精读/ 的英文写作规范
- **搭桥:** turbine-blade-ai-platform (应用章节); 人工智能基础与应用/课程轨/大论文 (一稿两用)

七、自检清单 (结项前必过, 抄自《判分与验收标准》§ 五)

1. 每个数字都能在 证据/ 找到原始日志吗?
2. “变好了” 有没有可能是口径变了 / 只挑了好种子?
3. 有没有把**训练集表现**当能力?
4. 每个“因为…所以…” 有没有反事实检验?
5. 图表的坐标轴与误差棒都标了吗?
6. 我写下的最强反方论点是什么?

八、风险与提醒

- ⚠ 报告最易犯的错是**夸大**: 规模受限就要说受限, 这反而是可信度来源。
- ⚠ 不要为了“好看”删掉 no-go 记录——用户明确要求“负结果不许擦除”。
- ⚠ W16 现场 demo 必须有**失败预案** (预生成结果 + 录屏), 不依赖现场网络与算力。